MARMARA
ÜNİVERSİTESİ

Introduction to Databases

Assoc.Prof.Selçuk KIRAN

Database System Project - 2. Phase

**Şerefim ve inandığım tüm değerler adına yemin ederim ki bu projede hiçbir yerden kopya
çekmedim ve copy paste yapmadım.**
**I swear, in the name of my honor and all the values I believed in, that I did not copy and paste
anything from anywhere during this project.**




Sevinç ÇAKAR (138723040)	Eylül ÇİMEN (138723515)	Nehir KARASAKAL (138722019)	Yasin SÜLEYMANOĞLU (138722008)

sevinç

1-SQL SELECTQUERIES

```sql
--1.Whichcustomer madehowmuch total payment? (GROUP BY)
SELECT C.Full_Name, SUM(R.Payment) AS Toplam_Odeme
FROM Customer C
JOIN Reservation RON C.Customer_ID = R.Customer_ID
GROUP BY C.Full_Name
ORDER BY Toplam_Odeme DESC;
```

Results Messages

	Full_Name	Toplam_Odeme
1	Musteri 103	500.00
2	Musteri 109	500.00
3	Musteri 118	500.00
4	Musteri 120	500.00
5	Musteri 137	500.00
6	Musteri 173	500.00
7	Musteri 2	500.00
8	Musteri 200	500.00
9	Musteri 41	500.00

Ln 1, Col 1 (234 selected) Spaces: 4 UTF-8 CRLF SQL 200 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 2. Which bicycle model was rented how many times? (JOIN + COUNT)
SELECT  M.Bicycle_Brand,  COUNT(R.Reservation_ID)  AS  Kiralama_Sayisi
FROM Bicycle_Model M JOIN Bicycle B ON M.Model_ID = B.Model_ID JOIN
Reservation R ON B.B_QR_Code = R.B_QR_Code
GROUP BY M.Bicycle_Brand;
```

Results Messages

	Results grid e_Brand	Kiralama_Sayisi
1	Bianchi City	64
2	Carraro E-Bike	63
3	Salcano Mountain	73

Ln 8, Col 1 (276 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 3. Insurance information of bicycles with 'Available' status only
SELECT B.B_QR_Code, I.Insurance_Name, I.Ins_Expiry_Date
FROM Bicycle B
JOIN Insurance I ON B.Ins_Policy_No = I.Ins_Policy_No
WHERE B.Status = 'Available';
```

Results Messages

	B_QR_Code	Insurance_Name	Ins_Expiry_Date
1	QR001	Allianz	2026-01-01
2	QR002	Axa Sigorta	2026-01-01
3	QR003	Allianz	2026-01-01
4	QR004	Axa Sigorta	2026-01-01
5	QR005	Allianz	2026-01-01
6	QR006	Axa Sigorta	2026-01-01
7	QR007	Allianz	2026-01-01
8	QR008	Axa Sigorta	2026-01-01
9	QR009	Allianz	2026-01-01

Ln 15, Col 1 (227 selected) Spaces: 4 UTF-8 CRLF SQL 30 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 4. Phone numbers of customers who made payments over 200 TL (WHERE)
```

```sql
SELECT C.Full_Name, C.Phone_Number, R.Payment
FROM Customer C
JOIN Reservation R ON C.Customer_ID = R.Customer_ID
WHERE R.Payment > 200;
```

Results Messages

	Full_Name	Phone_Number	Payment
1	Musteri 2	+905320000002	500.00
2	Musteri 4	+905320000004	300.00
3	Musteri 5	+905320000005	400.00
4	Musteri 9	+905320000009	225.00
5	Musteri 13	+905320000013	300.00
6	Musteri 15	+905320000015	250.00
7	Musteri 19	+905320000019	300.00
8	Musteri 22	+905320000022	300.00
9	Musteri 23	+905320000023	400.00

Ln 21, Col 1 (211 selected) Spaces: 4 UTF-8 CRLF SQL 108 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 5. What is the average rental fee for each bicycle class? (AVG)
SELECT BC.Price_Per_Day, AVG(R.Payment) AS Ortalama_Gelir
FROM Bicycle_Class BC
JOIN Bicycle_Model M ON BC.Bicycle_Class_ID = M.Bicycle_Class_ID
JOIN Bicycle B ON M.Model_ID = B.Model_ID
JOIN Reservation R ON B.B_QR_Code = R.B_QR_Code
GROUP BY BC.Price_Per_Day;
```

Results Messages

	Price_Per_Day	Ortalama_Gelir
1	50.00	160.156250
2	75.00	237.328767
3	100.00	307.936507

Ln 27, Col 1 (334 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 6. Customers with total payments exceeding 500 TL (HAVING)
SELECT C.Full_Name, SUM(R.Payment) AS Toplam
FROM Customer C
JOIN Reservation R ON C.Customer_ID = R.Customer_ID
GROUP BY C.Full_Name
HAVING SUM(R.Payment) > 500;
```

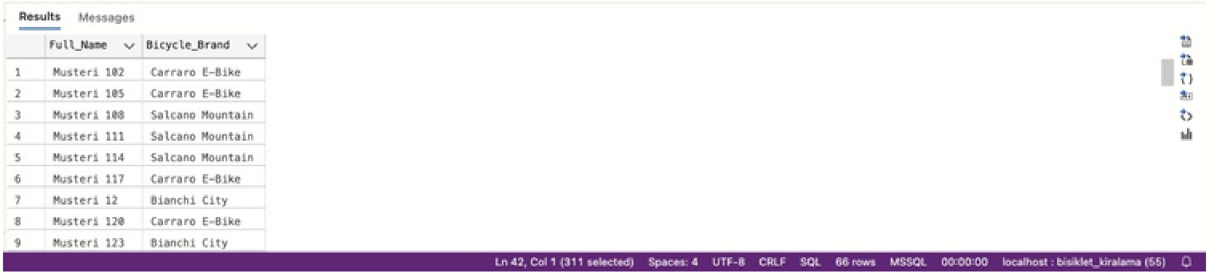
Results Messages

	Full_Name	Toplam

Ln 35, Col 1 (229 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

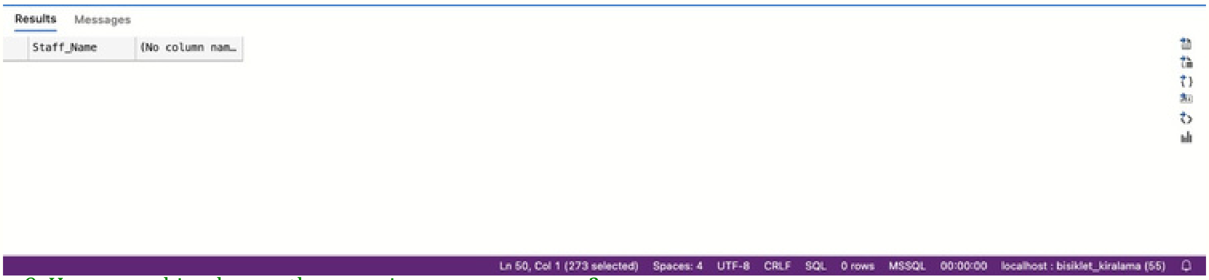
```sql
-- 7. Bicycle models rented by those paying with credit card
SELECT DISTINCT C.Full_Name, M.Bicycle_Brand
FROM Reservation R
JOIN Customer C ON R.Customer_ID = C.Customer_ID
JOIN Bicycle B ON R.B_QR_Code = B.B_QR_Code
JOIN Bicycle_Model M ON B.Model_ID = M.Model_ID
```

```sql
WHERE R.Payment_Method = 'Credit Card';
```



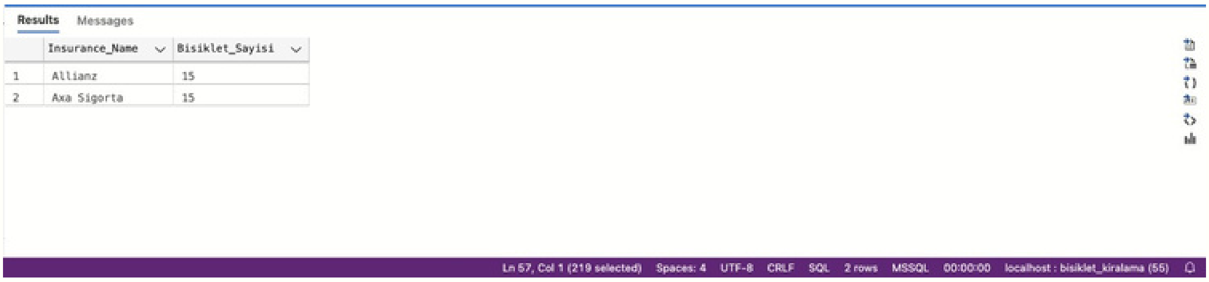
	Full_Name	Bicycle_Brand
1	Musteri 102	Carraro E-Bike
2	Musteri 105	Carraro E-Bike
3	Musteri 108	Salcano Mountain
4	Musteri 111	Salcano Mountain
5	Musteri 114	Salcano Mountain
6	Musteri 117	Carraro E-Bike
7	Musteri 12	Bianchi City
8	Musteri 120	Carraro E-Bike
9	Musteri 123	Bianchi City

```sql
-- 8. Staff member performing the most maintenance and the count
-- Note: This may return empty if Maintain table is not populated.
SELECT MP.Staff_Name, COUNT(M.Maintain_ID)
FROM Maintain_Personal MP
JOIN Maintain M ON MP.Staff_ID = M.Staff_ID
GROUP BY MP.Staff_Name;
```



Staff_Name	(No column nam…

```sql
-- 9. How many bicycles are there per insurance company?
SELECT I.Insurance_Name, COUNT(B.B_QR_Code) AS Bisiklet_Sayisi
FROM Insurance I
JOIN Bicycle B ON I.Ins_Policy_No = B.Ins_Policy_No
GROUP BY I.Insurance_Name;
```



	Insurance_Name	Bisiklet_Sayisi
1	Allianz	15
2	Axa Sigorta	15

```sql
-- 10. Reservations made between specific dates
SELECT R.Reservation_ID, C.Full_Name, R.Start_Time
FROM Reservation R
JOIN Customer C ON R.Customer_ID = C.Customer_ID
WHERE R.Start_Time BETWEEN '2025-11-01' AND '2025-12-31';
```

Results Messages

	Reservation_ID	Full_Name	Start_Time
1	1	Musteri 1	2025-11-13 15:35:20.623
2	2	Musteri 2	2025-11-05 15:35:20.630
3	3	Musteri 3	2025-11-06 15:35:20.630
4	4	Musteri 4	2025-11-10 15:35:20.633
5	5	Musteri 5	2025-11-06 15:35:20.633
6	6	Musteri 6	2025-11-07 15:35:20.633
7	7	Musteri 7	2025-11-06 15:35:20.637
8	8	Musteri 8	2025-11-04 15:35:20.637
9	9	Musteri 9	2025-11-15 15:35:20.633

Ln 63, Col 1 (228 selected) Spaces: 4 UTF-8 CRLF SQL 200 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 11. Which bicycle has never been rented? (LEFT JOIN - NULL check)
SELECT B.B_QR_Code
FROM Bicycle B
LEFT JOIN Reservation R ON B.B_QR_Code = R.B_QR_Code
WHERE R.Reservation_ID IS NULL;
```

Results Messages

	B_QR_Code

Results grid

Ln 68, Col 1 (195 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 12. Who is the customer with the highest payment? (TOP 1)
SELECT TOP 1 C.Full_Name, R.Payment
FROM Customer C
JOIN Reservation R ON C.Customer_ID = R.Customer_ID
ORDER BY R.Payment DESC;
```

Results Messages

	Full_Name	Payment
1	Musteri 2	500.00

Ln 75, Col 1 (193 selected) Spaces: 4 UTF-8 CRLF SQL 1 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 13. (EXCEPT) Customers who have never paid with "Cash" (always usedCredit Card or Mobile Pay)
-- Removes customers who used cash from the list of all renters (EXCEPT).
SELECT Customer_ID FROM Reservation
EXCEPT
SELECT Customer_ID FROM Reservation WHERE Payment_Method = 'Cash';
```

Results Messages

	Customer_ID
1	1
2	3
3	4
4	6
5	7
6	9
7	10
8	12
9	13

Ln 81, Col 1 (285 selected) Spaces: 4 UTF-8 CRLF SQL 133 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 14. (TOP & ORDER BY) List the top 5 reservations with the highest payments
-- No complex formula; simply sorts descending and takes the top 5.
SELECT TOP 5 Customer_ID, Payment, Payment_Method, Start_Time
FROM Reservation
ORDER BY Payment DESC;
```

Results Messages

	Customer_ID	Payment	Payment_Method	Start_Time
1	2	500.00	Cash	2025-11-05 15:35:20.630
2	41	500.00	Cash	2025-11-15 15:35:20.647
3	69	500.00	Credit Card	2025-11-14 15:35:20.683
4	71	500.00	Cash	2025-11-26 15:35:20.657
5	103	500.00	Mobile Pay	2025-11-05 15:35:20.717

Ln 87, Col 1 (251 selected) Spaces: 4 UTF-8 CRLF SQL 5 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 15. Reservations with a rental duration of more than 2 days
SELECT R.Reservation_ID, R.Time_Lag / 1440 AS Gun_Sayisi
FROM Reservation R
WHERE (R.Time_Lag / 1440) > 2;
```

Results Messages

	Reservation_ID	Gun_Sayisi
1	2	5
2	4	4
3	5	4
4	7	3
5	9	3
6	12	3
7	13	3
8	15	5
9	16	4

Ln 93, Col 1 (172 selected) Spaces: 4 UTF-8 CRLF SQL 133 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 16. (EXISTS) Is there any customer who hasn't made a rental?
SELECT Full_Name FROM Customer C
WHERE NOT EXISTS (SELECT 1 FROM Reservation R WHERE R.Customer_ID = C.Customer_ID);
```

Results Messages

Full_Name

Ln 98, Col 1 (184 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 17. (IN) Reservations of customers with IDs 1, 5, or 10 only
SELECT * FROM Reservation WHERE Customer_ID IN (1, 5, 10);
```

Results Messages

	Reservation_ID	Customer_ID	B_QR_Code	Start_Time	End_Time	Payment	Time_Lag	Payment_Method
1	1	1	QR009	2025-11-13 15:35:20.623	2025-11-14 15:35:20.623	75.00	1440	Mobile Pay
2	5	5	QR017	2025-11-06 15:35:20.633	2025-11-10 15:35:20.633	400.00	5760	Cash
3	10	10	QR011	2025-11-11 15:35:20.640	2025-11-12 15:35:20.640	100.00	1440	Mobile Pay

Ln 104, Col 1 (125 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 18. (UNION) Combine Customer names and Staff names into a single list
SELECT Full_Name AS Kisi_Listesi, 'Musteri' AS Tip FROM Customer
UNION
SELECT Staff_Name, 'Personel' FROM Maintain_Personal;
```

Results Messages

	Kisi_Listesi	Tip
1	Ahmet Yilmaz	Personel
2	Ayse Demir	Personel
3	Mehmet Kaya	Personel
4	Musteri 1	Musteri
5	Musteri 10	Musteri
6	Musteri 100	Musteri
7	Musteri 101	Musteri
8	Musteri 102	Musteri
9	Musteri 103	Musteri

Ln 104, Col 1 (204 selected) Spaces: 4 UTF-8 CRLF SQL 203 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 19. (INTERSECT) Is there a common ID in both Bicycle Models and Classes? (Technical query)
-- (Shows the command usage)
SELECT Model_ID FROM Bicycle_Model
INTERSECT
SELECT Bicycle_Class_ID FROM Bicycle_Class;
```

Results Messages

Model_ID

Ln 110, Col 1 (216 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 20. (EXCEPT) Customers who made reservations but received no penalties
SELECT Customer_ID FROM Customer
EXCEPT
SELECT C.Customer_ID FROM Customer C
JOIN Reservation R ON C.Customer_ID = R.Customer_ID
JOIN Penalty P ON R.Reservation_ID = P.Reservation_ID;
```

Results Messages

	Customer_ID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

Results grid

Ln 116, Col 1 (262 selected) Spaces: 4 UTF-8 CRLF SQL 200 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 21. (Subquery) Transactions with payments higher than the average
SELECT * FROM Reservation
WHERE Payment > (SELECT AVG(Payment) FROM Reservation);
```

Results Messages

	Reservation_ID	Customer_ID	B_QR_Code	Start_Time	End_Time	Payment	Time_Lag	Payment_Method
1	2	2	QR029	2025-11-05 15:35:20.630	2025-11-10 15:35:20.630	500.00	7200	Cash
2	4	4	QR024	2025-11-10 15:35:20.633	2025-11-14 15:35:20.633	300.00	5760	Mobile Pay
3	5	5	QR017	2025-11-06 15:35:20.633	2025-11-10 15:35:20.633	400.00	5760	Cash
4	13	13	QR002	2025-11-12 15:35:20.640	2025-11-15 15:35:20.640	300.00	4320	Mobile Pay
5	15	15	QR004	2025-11-11 15:35:20.637	2025-11-16 15:35:20.637	250.00	7200	Credit Card
6	19	19	QR030	2025-11-06 15:35:20.647	2025-11-10 15:35:20.647	300.00	5760	Mobile Pay
7	22	22	QR018	2025-11-11 15:35:20.650	2025-11-15 15:35:20.650	300.00	5760	Mobile Pay
8	23	23	QR008	2025-11-16 15:35:20.647	2025-11-20 15:35:20.647	400.00	5760	Cash
9	24	24	QR020	2025-11-11 15:35:20.650	2025-11-14 15:35:20.650	300.00	4320	Credit Card

Ln 123, Col 1 (153 selected) Spaces: 4 UTF-8 CRLF SQL 93 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 22. (LIKE) Customers whose names start with 'A'
SELECT * FROM Customer WHERE Full_Name LIKE 'A%';
```

Results Messages

Customer_ID	Full_Name	Phone_Number	E_Mail

Ln 127, Col 1 (101 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 23. (NOT EQUAL) List transactions where the payment method is not 'Cash'
-- Uses only the "<>" (Not Equal) operator.
SELECT Reservation_ID, Customer_ID, Payment, Payment_Method
FROM Reservation
WHERE Payment_Method <> 'Cash';
```

Results Messages

	Reservation_ID	Customer_ID	Payment	Payment_Method
1	1	1	75.00	Mobile Pay
2	3	3	75.00	Credit Card
3	4	4	300.00	Mobile Pay
4	6	6	150.00	Credit Card
5	7	7	150.00	Mobile Pay
6	9	9	225.00	Credit Card
7	10	10	100.00	Mobile Pay
8	12	12	150.00	Credit Card
9	13	13	300.00	Mobile Pay

Results grid

Ln 130, Col 1 (234 selected) Spaces: 4 UTF-8 CRLF SQL 133 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 24. (DISTINCT) List different payment methods
SELECT DISTINCT Payment_Method FROM Reservation;
```

Results Messages

	Payment_Method
1	Cash
2	Credit Card
3	Mobile Pay

Ln 136, Col 1 (98 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 25. (DATE FUNCTIONS) Find the day of the week the rental was made (Monday, Tuesday..)
SELECT Reservation_ID, DATENAME(WEEKDAY, Start_Time) as Gun FROM Reservation;
```

Results Messages

	Reservation_ID	Gun
1	1	Thursday
2	2	Wednesday
3	3	Thursday
4	4	Monday
5	5	Thursday
6	6	Friday
7	7	Thursday
8	8	Tuesday
9	9	Saturday

Ln 139, Col 1 (169 selected) Spaces: 4 UTF-8 CRLF SQL 200 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 26. (HAVING) "Loyal Customers" who rented more than once
-- Usage of GROUP BY and HAVING.
SELECT Customer_ID, COUNT(*) AS Toplam_Kiralama
FROM Reservation
GROUP BY Customer_ID
HAVING COUNT(*) > 1;
```

Results Messages

Customer_ID	Toplam_Kiralama

Ln 141, Col 1 (206 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 27. (UNION) Customer and Staff Email List (For mass communication)
-- Combines data from two different tables into one list (UNION).
SELECT E_Mail, 'Musteri' AS Rol FROM Customer
UNION
SELECT E_Mail, 'Personel' AS Rol FROM Maintain_Personal;
```

Results Messages

	E_Mail	Rol
1	ahmet@bikerent.com	Personel
2	ayse@bikerent.com	Personel
3	mehmet@bikerent.com	Personel
4	musteri1@mail.com	Musteri
5	musteri10@mail.com	Musteri
6	musteri100@mail.com	Musteri
7	musteri101@mail.com	Musteri
8	musteri102@mail.com	Musteri
9	musteri103@mail.com	Musteri

Ln 149, Col 1 (248 selected) Spaces: 4 UTF-8 CRLF SQL 203 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 28. (INTERSECT) Customers who used both Cash and Credit Card
-- Takes the intersection of two sets (INTERSECT).
SELECT Customer_ID FROM Reservation WHERE Payment_Method = 'Cash'
INTERSECT
SELECT Customer_ID FROM Reservation WHERE Payment_Method = 'Credit Card';
```

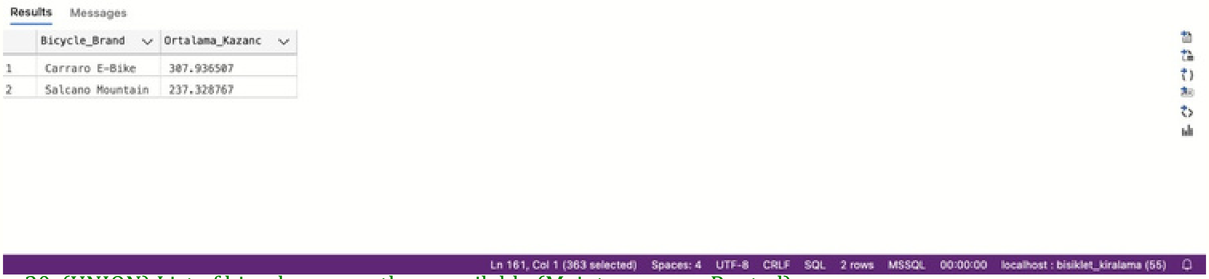
Results Messages

Customer_ID

Ln 155, Col 1 (268 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 29. (HAVING + JOIN) Bicycle brands with average revenue higher than 200 TL
-- Grouping and condition setting after Join operation.
SELECT M.Bicycle_Brand, AVG(R.Payment) AS Ortalama_Kazanc
FROM Bicycle_Model M
JOIN Bicycle B ON M.Model_ID = B.Model_ID
JOIN Reservation R ON B.B_QR_Code = R.B_QR_Code
```

```sql
GROUP BY M.Bicycle_Brand
HAVING AVG(R.Payment) > 200;
```

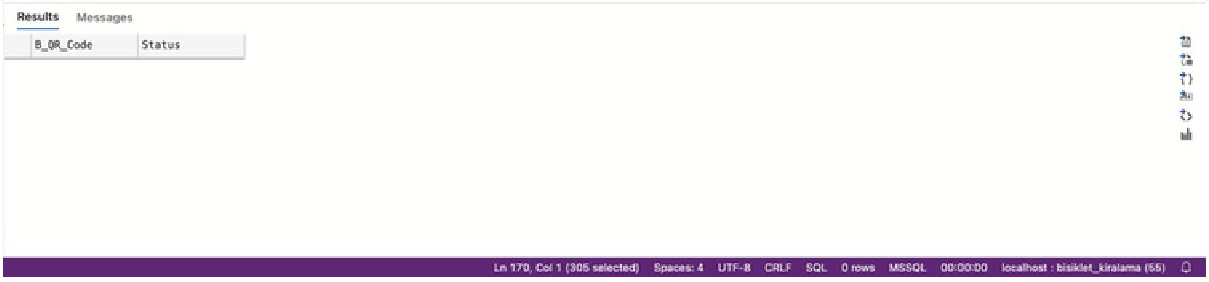


Results Messages

	Bicycle_Brand	Ortalama_Kazanc
1	Carraro E-Bike	307.936507
2	Salcano Mountain	237.328767

Ln 161, Col 1 (363 selected) Spaces: 4 UTF-8 CRLF SQL 2 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

```sql
-- 30. (UNION) List of bicycles currently unavailable (Maintenance or Rented)
-- Collects bicycles in Maintenance and Rented status into a single "Unavailable" list.
SELECT B_QR_Code, Status FROM Bicycle WHERE Status = 'Maintenance'
UNION
SELECT B_QR_Code, Status FROM Bicycle WHERE Status = 'Rented';
```



Results Messages

	B_QR_Code	Status

Ln 170, Col 1 (305 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (55)

2-INSERT – UPDATE –DELETE

```sql
-- --- 5 INSERT STATEMENTS (Adding New Data) ---
-- 1. Adding a new maintenance staff member
INSERT INTO Maintain_Personal (Staff_ID, Staff_Name, Phone, E_Mail, Role)
VALUES (4, 'Canan Yıldız', '+905320000099', 'canan@bikerent.com', 'Technician');

-- 2. Adding a new insurance company
INSERT INTO Insurance (Ins_Policy_No, Insurance_Name, Ins_Begin_Date, Ins_Expiry_Date, Price_Per_Year)
VALUES ('POL999', 'Mapfre Sigorta', '2025-06-01', '2026-06-01', 550);

-- 3. Adding a new category to the Bicycle Class
INSERT INTO Bicycle_Class (Bicycle_Class_ID, Price_Per_Day)
VALUES (4, 150.00);

-- 4. Adding a new bicycle type belonging to that category
INSERT INTO Bicycle_Type (B_Type_ID, B_Type_Name, Bicycle_Class_ID)
VALUES (4, 'Tandem', 4);

-- 5. Adding a new model
INSERT INTO Bicycle_Model (Model_ID, Bicycle_Class_ID, Bicycle_Brand, B_Seat_Capacity, B_Wheel_Count)
VALUES (104, 4, 'Salcano Tandem', 2, 2);
```

```sql
-- --- 5 UPDATE STATEMENTS (Updating Data) ---
-- 1. Updating a staff member's phone number
UPDATE Maintain_Personal SET Phone = '+905321112233' WHERE Staff_ID = 1;
-- 2. Changing a bicycle's status to 'Maintenance'
UPDATE Bicycle SET Status = 'Maintenance' WHERE B_QR_Code = 'QR005';
-- 3. Increasing insurance prices by 10%
UPDATE Insurance
SET Price_Per_Year = Price_Per_Year * 1.10;
-- 4. Changing a specific customer's full name (e.g., due to marriage)
UPDATE Customer SET Full_Name = 'Ayşe Yılmaz' WHERE Customer_ID = 2;
-- 5. Updating the daily rental price of electric bicycles
UPDATE Bicycle_Class SET Price_Per_Day = 120.00 WHERE Bicycle_Class_ID = 2;
```

```sql
-- --- 5 DELETE STATEMENTS (Deleting Data) --- -- Note: We delete child
records first to avoid referential integrity errors. -- 1. Deleting the Tandem
model added earlier (No bicycles attached yet) DELETE FROM
Bicycle_Model WHERE Model_ID = 104;
-- 2. Deleting the bicycle type added earlier
DELETE FROM Bicycle_Type WHERE B_Type_ID = 4;
-- 3. Deleting the class added earlier
DELETE FROM Bicycle_Class WHERE Bicycle_Class_ID =4;

-- 4. Deleting an expired/old insurance record (Example)
DELETE FROM Insurance WHERE Ins_Policy_No = 'POL999';
-- 5. Deleting a staff member who left the job (Staff ID 4)
DELETE FROM Maintain_Personal WHERE Staff_ID = 4;
```

3-ALTER TABLE – DROP TABLE

```sql
-- --- 5 ALTER TABLE STATEMENTS (Modifying Table Structure) ---

-- 1. Adding an 'Address' column to the Customer table
ALTER TABLE Customer ADD Address VARCHAR(255);
-- 2. Adding a 'Birth Date' column to the Staff table
ALTER TABLE Maintain_Personal ADD Birth_Date DATE;
-- 3. Increasing the character limit of the 'Status' field in the Bicycle table
ALTER TABLE Bicycle ALTER COLUMN Status VARCHAR(20);
-- 4. Adding an 'Agent Phone' column to the Insurance table
ALTER TABLE Insurance ADD Agent_Phone VARCHAR(15);
```

```
-- 5. Adding a 'Gear Count' column to the Bicycle Modeltable
ALTER TABLE Bicycle_Model ADD Gear_Count TINYINT;



-- --- 5 DROP TABLE STATEMENTS (Deleting Tables) --- -- First, let's
create temporary tables to delete: CREATE TABLE Test_Table1 (ID INT);
CREATE TABLE Test_Table2 (ID INT); CREATE TABLE Test_Table3 (ID
INT);
CREATE TABLE Test_Table4 (ID INT);
CREATE TABLE Test_Table5 (ID INT);

-- Now, let's delete them one by one (Include these codes in the report):
DROP TABLE Test_Table1;
DROP TABLE Test_Table2;
DROP TABLE Test_Table3;
DROP TABLE Test_Table4;
DROP TABLE Test_Table5;
```

4-INDEX

```
-- 1. To quickly search for customers by their full name
CREATE INDEX idx_customer_name ON Customer(Full_Name);

-- 2. To filter bicycles based on their status (Available/Rented)
CREATE INDEX idx_bicycle_status ON Bicycle(Status);
-- 3. To report reservations based on the start date
CREATE INDEX idx_reservation_start ON Reservation(Start_Time);
-- 4. To search for customers using their email address
CREATE INDEX idx_customer_email ON Customer(E_Mail);
-- 5. To group staff members based on their role
CREATE INDEX idx_staff_role ON Maintain_Personal(Role);
-- 6. To search for bicycles by their brand
CREATE INDEX idx_model_brand ON Bicycle_Model(Bicycle_Brand);



-- 7. To track insurance policies based on their expiry date
CREATE INDEX idx_insurance_expiry ON Insurance(Ins_Expiry_Date);
```

```sql
-- 8. To analyze data based on the payment method
CREATE INDEX idx_payment_method ON Reservation(Payment_Method);

-- 9. To sort maintenance records by date
-- Even if the Maintain table is currently empty, the index can still be created.
CREATE INDEX idx_maintain_date ON Maintain(Maintain_Date);

-- 10. To search for bicycle types by name
CREATE INDEX idx_type_name ON Bicycle_Type(B_Type_Name);
```

5-VIEW

```sql
-- 1. Active Rentals (Currently rented out)
CREATE VIEW View_Active_Rentals AS
SELECT R.Reservation_ID, C.Full_Name, B.B_QR_Code, R.Start_Time
FROM Reservation R
JOIN Customer C ON R.Customer_ID = C.Customer_ID
JOIN Bicycle B ON R.B_QR_Code = B.B_QR_Code
WHERE R.End_Time > GETDATE();

SELECT * FROM View_Active_Rentals;
```

Results Messages

Reservation_ID	Full_Name	B_QR_Code	Start_Time

Ln 9, Col 1 (34 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 2. Detailed List of Bicycles (With Brand and Class information)
CREATE VIEW View_Bicycle_Details AS
SELECT B.B_QR_Code, M.Bicycle_Brand, C.Price_Per_Day, B.Status
FROM Bicycle B
JOIN Bicycle_Model M ON B.Model_ID = M.Model_ID
JOIN Bicycle_Class C ON M.Bicycle_Class_ID = C.Bicycle_Class_ID;

SELECT * FROM View_Bicycle_Details;
```

Results Messages

Show Filter

	B_QR_Code	Bicycle_Brand	Price_Per_Day	Status
1	QR001	Bianchi City	50.00	Available
2	QR002	Carraro E-Bike	100.00	Available
3	QR003	Salcano Mountain	75.00	Available
4	QR004	Bianchi City	50.00	Available
5	QR005	Carraro E-Bike	100.00	Available
6	QR006	Salcano Mountain	75.00	Available
7	QR007	Bianchi City	50.00	Available
8	QR008	Carraro E-Bike	100.00	Available

Ln 17, Col 1 (39 selected) Spaces: 4 UTF-8 CRLF SQL 30 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 3. High Turnover Customers (VIP)
CREATE VIEW View_VIP_Customers3 AS
SELECT C.Full_Name, SUM(R.Payment) as Total_Payment
FROM Customer C
JOIN Reservation R ON C.Customer_ID = R.Customer_ID
GROUP BY C.Customer_ID, C.Full_Name;

SELECT * FROM View_VIP_Customers3 ORDER BY Total_Payment DESC;
```

Results Messages

	Full_Name	Total_Payment
1	Musteri 2	500.00
2	Musteri 41	500.00
3	Musteri 69	500.00
4	Musteri 71	500.00
5	Musteri 103	500.00
6	Musteri 109	500.00
7	Musteri 118	500.00
8	Musteri 120	500.00

Ln 27, Col 1 (62 selected) Spaces: 4 UTF-8 CRLF SQL 200 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 4. Bicycles Needing Maintenance
CREATE VIEW View_Maintenance_Needed AS
SELECT B_QR_Code, Status, Ins_Policy_No
FROM Bicycle
WHERE Status = 'Maintenance';

SELECT * FROM View_Maintenance_Needed;
```

Results Messages

B_QR_Code	Status	Ins_Policy_No

Ln 35, Col 1 (38 selected) Spaces: 4 UTF-8 CRLF SQL 0 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 5. Insurances About to Expire
CREATE VIEW View_Expiring_Insurances AS
SELECT Ins_Policy_No, Insurance_Name, Ins_Expiry_Date
FROM Insurance
WHERE Ins_Expiry_Date < DATEADD(MONTH, 3, GETDATE());

SELECT * FROM View_Expiring_Insurances;
```

Results Messages

	Ins_Policy_No	Insurance_Name	Ins_Expiry_Date
1	POL001	Axa Sigorta	2026-01-01
2	POL002	Allianz	2026-01-01

Ln 42, Col 1 (41 selected) Spaces: 4 UTF-8 CRLF SQL 2 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 6. Staff Contact List
CREATE VIEW View_Staff_Contact AS
SELECT Staff_Name, Role, Phone, E_Mail FROM Maintain_Personal;

SELECT * FROM View_Staff_Contact;
```

Results Messages

	Staff_Name	Role	Phone	E_Mail
1	Ahmet Yilmaz	Manager	+905320000001	ahmet@bikerent.com
2	Ayse Demir	Technician	+905320000002	ayse@bikerent.com
3	Mehmet Kaya	Staff	+905320000003	mehmet@bikerent.com

Ln 49, Col 1 (33 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 7. Electric Bicycle Rental History
CREATE VIEW View_Electric_Rentals AS
SELECT R.Reservation_ID, B.B_QR_Code, R.Start_Time
FROM Reservation R
JOIN Bicycle B ON R.B_QR_Code = B.B_QR_Code
JOIN Bicycle_Model M ON B.Model_ID = M.Model_ID
JOIN Bicycle_Class C ON M.Bicycle_Class_ID = C.Bicycle_Class_ID
WHERE C.Price_Per_Day > 80; -- Prices over 80 are assumed to be electric

SELECT * FROM View_Electric_Rentals;
```

Results Messages

	Reservation_ID	B_QR_Code	Start_Time
1	2	QR029	2025-11-05 15:35:20.630
2	5	QR017	2025-11-06 15:35:20.633
3	10	QR011	2025-11-11 15:35:20.640
4	13	QR002	2025-11-12 15:35:20.640
5	17	QR026	2025-11-13 15:35:20.647
6	18	QR008	2025-11-11 15:35:20.647
7	23	QR008	2025-11-16 15:35:20.647
8	24	QR020	2025-11-11 15:35:20.650

Ln 60, Col 1 (38 selected) Spaces: 4 UTF-8 CRLF SQL 63 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 8. Daily Income Analysis
CREATE VIEW View_Daily_Income AS
SELECT CAST(Start_Time AS DATE) as Rental_Day, SUM(Payment) as Daily_Revenue
FROM Reservation
GROUP BY CAST(Start_Time AS DATE);

SELECT * FROM View_Daily_Income;
```

Results Messages

	Rental_Day	Daily_Revenue
1	2025-11-04	225.00
2	2025-11-05	1175.00
3	2025-11-06	1150.00
4	2025-11-07	625.00
5	2025-11-08	1425.00
6	2025-11-09	1025.00
7	2025-11-10	1050.00
8	2025-11-11	1975.00

Ln 68, Col 1 (32 selected) Spaces: 4 UTF-8 CRLF SQL 49 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 9. Bicycle Counts by Brand
CREATE VIEW View_Brand_Counts AS
SELECT Bicycle_Brand, COUNT(*) as Bike_Count
FROM Bicycle_Model
GROUP BY Bicycle_Brand;

SELECT * FROM View_Brand_Counts;
```

Results Messages

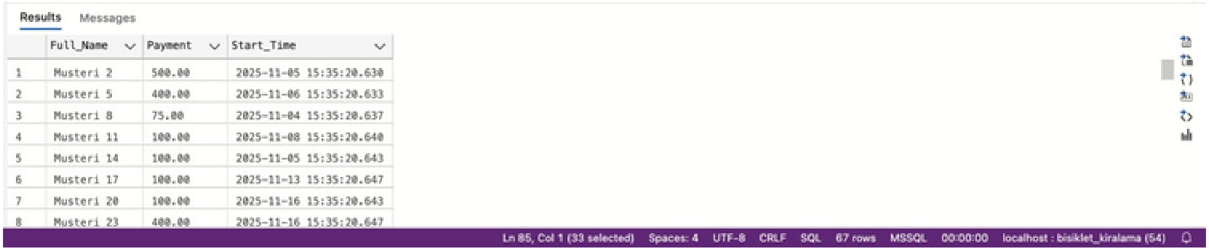
	Bicycle_Brand	Bike_Count
1	Bianchi City	1
2	Carraro E-Bike	1
3	Salcano Mountain	1

Ln 76, Col 1 (32 selected) Spaces: 4 UTF-8 CRLF SQL 3 rows MSSQL 00:00:00 localhost : bisiklet_kiralama (54)

```sql
-- 10. Customers Paying with Cash
```

```sql
CREATE VIEW View_Cash_Payments AS
SELECT C.Full_Name, R.Payment, R.Start_Time
FROM Reservation R
JOIN Customer C ON R.Customer_ID = C.Customer_ID
WHERE R.Payment_Method = 'Cash';

SELECT * FROM View_Cash_Payments;
```



Results Messages

	Full_Name	Payment	Start_Time
1	Musteri 2	500.00	2025-11-05 15:35:20.630
2	Musteri 5	400.00	2025-11-06 15:35:20.633
3	Musteri 8	75.00	2025-11-04 15:35:20.637
4	Musteri 11	100.00	2025-11-08 15:35:20.640
5	Musteri 14	100.00	2025-11-05 15:35:20.643
6	Musteri 17	100.00	2025-11-13 15:35:20.647
7	Musteri 20	100.00	2025-11-16 15:35:20.643
8	Musteri 23	400.00	2025-11-16 15:35:20.647